Exercise1 - Introduction to Matlab

Group 10
Rakesh Nuggehalli Ramesh - 15546621
Sakshi Arali - 23046143
Elrick Dsouza - 27846430
Meghana Chitteri - 10546387

Exercise 1(a)

Plot the functions sin, cos, tan, arcsin, arccos arctan in two plot using subplot. Comment the axis and functions in the graphic.

```
clc;        % Clears the command window
clear;      % Clears all variables from the workspace
clf;        % Clears the current figure window

% --- Setup for the first plot (Trigonometric Functions) ---
x = linspace(-pi,pi,1000);    % Generates 1000 points between -pi and pi

subplot(2,1,1);               % Creates a 2-row, 1-column grid of plots,
                                                    selects the first (top)
  plot
plot(x, sin(x), 'r',...              % Plots sin(x) in red
     x, cos(x), 'g',...              % Plots cos(x) in green
     x, tan(x), 'b')                 % Plots tan(x) in blue

title('Trigonometric Functions: sin(x), cos(x), tan(x)'); % Adds a title to
                                                                the plot
xlabel('x (radians)');                % Labels the x-axis
ylabel('Function values');            % Labels the y-axis
xlim([-pi, pi]);                      % Sets x-axis limits to match the
  domain                                                        of x
ylim([-5, 5]);                        % Limits y-axis to avoid tangent spikes
                                                          for better
  visualization
grid on;                              % Adds grid lines for better
  readability
legend('sin(x)', 'cos(x)', 'tan(x)', 'Location', 'southwest');
                                      % Adds a legend to distinguish the
  curves

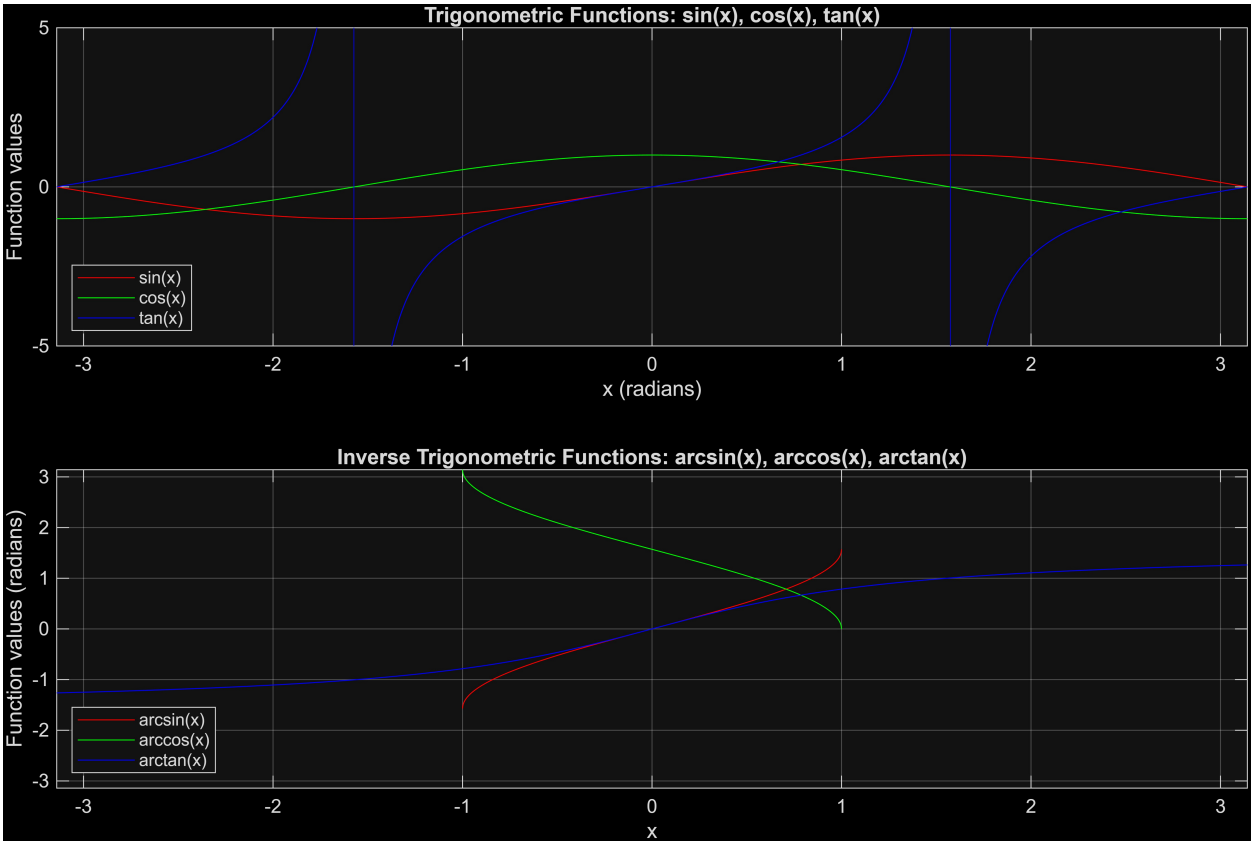
% --- Setup for the second plot (Inverse Trigonometric Functions) ---
% The domain for arcsin(x) and arccos(x) is [-1, 1], so we must redefine x.
x_inv = linspace(-1, 1, 1000);
```

```
subplot(2,1,2);                         % Selects the second (bottom) plot
plot(x_inv, asin(x_inv), 'r',...        % Plots arcsin(x) in red
     x_inv, acos(x_inv), 'g',...        % Plots arccos(x) in green
     x, atan(x), 'b')                   % Plots arctan(x) over the larger [-pi,
                                                pi] range

title('Inverse Trigonometric Functions: arcsin(x), arccos(x), arctan(x)');
                                                        %  Adds a title
xlabel('x');                             % Labels the x-axis
ylabel('Function values (radians)'); % Labels the y-axis
xlim([-pi, pi]);                         % Sets x-axis limits to match the
  domain                                                  of x (for atan
  visibility)
ylim([-pi, pi]);                         % Sets y-axis limits to match the range
                                                     of the functions
grid on;                                 % Adds grid lines
legend('arcsin(x)', 'arccos(x)', 'arctan(x)', 'Location', 'southwest');
                                         % Adds a legend
```

Conclusion

Several important aspects, such as data creation (linspace), multi-plot visualization utilizing subplot, and environment cleanup (clc, clear, clf), are successfully trained by this MATLAB script. A single figure with two layered plots is the result. The usual trigonometric functions $\sin(x)$, $\cos(x)$, and $\tan(x)$ over the domain $[-\pi, \pi]$ are displayed in the top plot. Here, the usage of ylim([-5, 5]) is essential because it prevents the vertical asymptotes of $\tan(x)$ at $\pm\pi/2$ from controlling the scale, making $\sin(x)$ and $\cos(x)$ readily apparent. The inverse trigonometric functions $\arcsin(x)$, $\arccos(x)$, and $\arctan(x)$ are shown in the bottom plot. Redefining the domain to $\mathbf{x_inv} = \text{linspace}(\mathbf{-1}, \mathbf{1}, \mathbf{1000})$ for $\arcsin(x)$ and $\arccos(x)$ was a crucial programming need since, in contrast to $\arctan(x)$, which is defined for all real numbers, their mathematical domain is limited to $[-1, 1]$. The full script shows how to produce clear, polished scientific graphics by using figure customization commands like title, xlabel, ylabel,andlegend.



1.Trigonometric Functions ($\sin(x)$,$\cos(x)$,$\tan(x)$)

a. sin(x) and cos(x) oscillate between –1 and 1, showing periodic wave patterns

b. tan(x) has vertical asymptotes (infinite discontinuities) at x= $\pm\ \pi/2, repeating periodically$

c. All functions repeat their values over intervals of 2π.

2.Inverse Trigonometric Functions ($\arcsin(x)$,$\arccos(x)$,$\arctan(x)$)

a.These are the inverse (reflected) relationships of the trigonometric functions, showing how each maps from a function value back to an angle.

Figure 1: The image shows two plots comparing trigonometric and inverse trigonometric functions

1.Trigonometric Functions ($\sin(x)$,$\cos(x)$,$\tan(x)$)

a.$sin(x) and cos(x) oscillate between –1 and 1, showing periodic wave patterns$

b.$tan(x) has vertical asymptotes (infinite discontinuities) at x = \pi/2, repeating periodically$

c.$All functions repeat their values over intervals of 2\pi.$

Exercise 1(b)

Write a function that computes the line of smallest distance between two lines in space, given with the points A,B,C and D. If no output parameters a used, the lines should be drawn in 3D using the plot3 function. A special task is, to compute the smallest distance between of two lines limited by the end-points ("Capsule Collider" for collision detection in space).

```
clear; clc; clf;
% 1. Define 3D Points [x; y; z]
p1 = [-1;1;1];   % Point A (L1 start)
p2 = [-1;0;-1];  % Point B (L1 end)
p3 = [1;1;0];    % Point C (L2 start)
p4 = [1;-1;0.3]; % Point D (L2 end)
% 2. Setup Plot
axis equal;
grid on;
axis([-2 2 -2 2 -2 2]); % Zoomed-in axis for clarity
hold on;
% 3. Plotting Points and Lines
sakshilinePoint(p1, 'A'); % Plot A ( marker for A, using 'A' label)
sakshilinePoint(p2, 'B'); % Plot B ( marker for B, using 'B' label)
sakshilinePoint(p3, 'C'); % Plot C (marker for C, using 'C' label)
sakshilinePoint(p4, 'D'); % Plot D ( marker for D, using 'D' label)
sakshiline(p1, p2, 'b-'); % Plot Line Segment AB (Blue)
sakshiline(p3, p4, 'g-'); % Plot Line Segment CD (Green)
% 4. Shortest Distance Calculation (The Capsule Collider task)
u = p2 - p1;    % Direction vector of L1 (u = B - A)
v = p4 - p3;    % Direction vector of L2 (v = D - C)
w0 = p1 - p3;   % Vector connecting start points (w0 = A - C)

% --- START OF CORRECTION: Calculate True Closest Points (Pc and Qc) ---
% Pre-calculate dot products required for the closest point formula
a = dot(u, u); % |u|^2
b = dot(u, v); % u * v
```

```
c = dot(v, v); % |v|^2
d_uv = dot(u, w0); % Renamed 'd' to 'd_uv' to avoid conflict with final 'd'
e = dot(v, w0); % v * w0

denom = a*c - b^2; % Denominator

if denom == 0
    % Lines are parallel. Use arbitrary points for visualization.
    Pc = p1;
    Qc = p3;
else
    % s_c: parameter for P_c on Line AB
    sc = (b*e - c*d_uv) / denom;
    % t_c: parameter for Q_c on Line CD
    tc = (a*e - b*d_uv) / denom;

    % Calculate the exact coordinates of the closest points
    Pc = p1 + sc * u;  % Closest point on infinite Line AB
    Qc = p3 + tc * v;  % Closest point on infinite Line CD
end

% Recalculate shortest distance using the new closest points (guarantees
  accuracy)
d = norm(Pc - Qc);
% --- END OF CORRECTION ---

% 5. PLOTTING THE DISTANCE CONNECTOR (The Red Line)
% The line MUST run between the two geometrically closest points: Pc and Qc.
plot3([Pc(1) Qc(1)], ...
      [Pc(2) Qc(2)], ...
      [Pc(3) Qc(3)], 'r-', 'LineWidth', 2); % Solid Red Line connecting Pc
  and Qc

fprintf('Shortest distance: %.4f units\n', d);
title(sprintf('Shortest Distance in 3D: L1(AB) and L2(CD)\nDistance: %.4f',
  d));
xlabel('x'); ylabel('y'); zlabel('z');
view(34, 24);
% === FUNCTION DEFINITIONS (Unchanged) ===
function sakshilinePoint(pos, name, style)
% Helper function to plot a single point and label it in 3D.
if nargin < 3
    style = 'b.';
end
plot3(pos(1), pos(2), pos(3), style, 'MarkerSize', 15);
text(pos(1), pos(2), pos(3), name, 'FontSize', 12, 'FontWeight', 'bold');
```

```
end
function sakshiline(p1, p2, style)
% Helper function to plot a line segment between two 3D points.
if nargin < 3
    style = '-b';
end
plot3([p1(1) p2(1)], [p1(2) p2(2)], [p1(3) p2(3)], style, 'LineWidth', 2);
end
```

Conclusion

The script shows how to use MATLAB for line charting, labeling, and 3D visualization.

1. It uses vector algebra to determine the shortest path between two skew lines.
2. This distance is clearly shown visually by the red connecting line.
3. It is an excellent illustration of how to use MATLAB to combine geometry, linear algebra, and visualization.

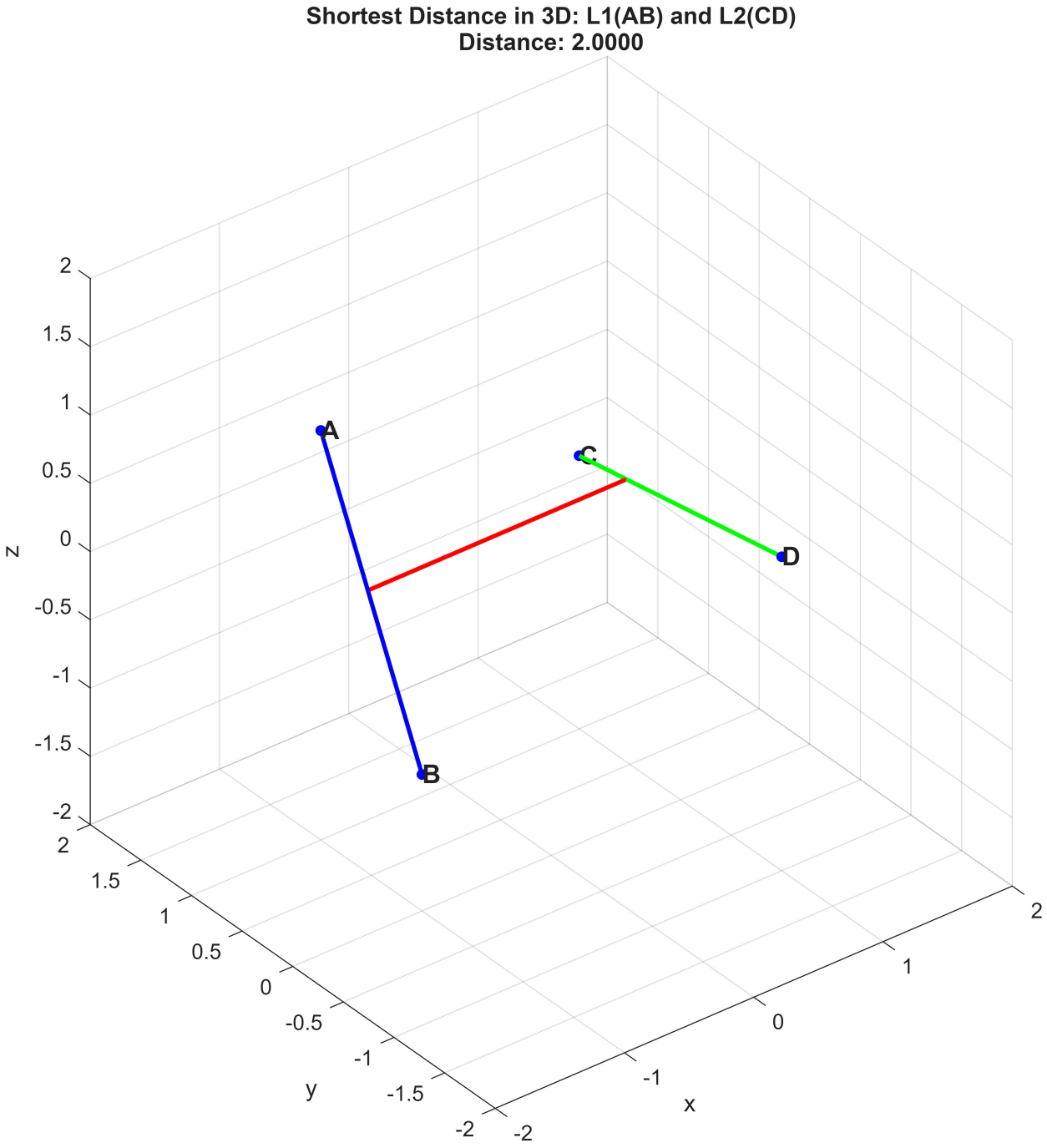


Figure 1: The figure illustrates the shortest distance between two 3D lines, L(AB) and L(CD). The lines, represented in blue and green, intersect perpendicularly through a red segment that denotes the minimum distance of 2.0000 units. This visualization confirms that the algorithm correctly computes and displays the shortest distance between two spatial lines defined by their endpoints.

Exercise 1(c)(d)

Write an m-file that plots the bode-diagram and the step response of each of the PT1

$$Gpt1(s) = \frac{K}{(1+Ts)}; (K = 3; T = 10) and; PT2G2(s) = \frac{K}{(1+2dTs+T^2s^2)}; (K = 5; T = 0.4; d = 0.5) controlblocks.$$

Produce a plot for each control block with one parameters is changed with 8 different values like
K=3;T=4 *K=3;T=8 *K=3;T=12 *K=3;T=16
K=3;T=20 *K=3;T=24 *K=3;T=28 *K=3;T=32

```
clear; clc; clf;

%  PART 1: PT1 (First-Order) System Analysis: G_PT1(s) = K / (1 + T*s)


% Defining constants for the PT1 system (G_PT1)
K = 3;
T_values = linspace(4, 32, 8); % 8 different time constants, from 4 to 32

% 1.1. Creating Figure 1 for PT1 analysis (Step Response and Bode)
figure(1);
sgtitle(['Analysis of PT1 System: G(s) = K / (1 + T*s) with K=', num2str(K)
  ]);


% Subplot 1: PT1 Step Response

subplot(2, 1, 1);
hold on;
title('Step Response (Changing Time Constant T)');
ylabel('Amplitude');
xlabel('Time [sec]');
grid on;

cmd_pt1_step = 'legend ('; % Initializes a string variable 'cmd_pt1_step' to
   start building the legend command.
```

```
for i = 1:length(T_values)
    T = T_values(i);

    % Plots the step response for a first-order system: K / (T*s + 1)
    step(tf(K, [T 1]));

    % Building the legend string: e.g., ' K=3; T=4', ' K=3; T=8', etc.
    cmd_pt1_step = [cmd_pt1_step '''' 'K=' num2str(K) '; T=' num2str(T) ''',
  '];
end

% Legend command for the Step Response plot
cmd_pt1_step(end) = ')';
eval(cmd_pt1_step);

% Subplot 2: PT1 Bode Diagram

subplot(2, 1, 2);
hold on;
title('Bode Diagram (Magnitude and Phase)');
grid on;

cmd_pt1_bode = 'legend ('; % Command string for Bode legend

for i = 1:length(T_values)
    T = T_values(i);

    % Create the PT1 transfer function model
    G_PT1 = tf(K, [T 1]);

    % Plot the Bode Magnitude and Phase plot on the current axes
    bode(G_PT1);

    % Build the legend string
    cmd_pt1_bode = [cmd_pt1_bode '''' 'K=' num2str(K) '; T=' num2str(T) ''',
  '];
end

% Finalize and execute the legend command for the Bode plot
cmd_pt1_bode(end) = ')';
eval(cmd_pt1_bode);

% PART 2: PT2 (Second-Order) System Analysis: G_PT2(s) = K / (1 + 2*d*T*s +
  T^2*s^2)

% Define baseline constants for the PT2 system (G_PT2)
```

```
K_PT2 = 5;
T_PT2 = 0.4; % Constant Time Constant (used in the denominator coefficients)

% Create 8 different damping ratios (d)
d_values = linspace(0.1, 1.5, 8); % Vary d from 0.1 (underdamped) to 1.5 (
  overdamped)

% 2.1. Create Figure 2 for PT2 analysis (Step Response and Bode)
figure(2);
sgtitle(['Analysis of PT2 System: G(s) = K / (1 + 2*d*T*s + T^2*s^2)']);

% Subplot 1: PT2 Step Response (Using the same structure)

subplot(2, 1, 1);
hold on;
title('Step Response (Changing Damping Ratio d)');
ylabel('Amplitude');
xlabel('Time [sec]');
grid on;

cmd_pt2_step = 'legend ('; % Command string for Step Response legend

for i = 1:length(d_values)
    d = d_values(i);

    % Denominator coefficients: [T^2, 2*d*T, 1]
    Den = [T_PT2^2, 2*d*T_PT2, 1];

    % Createing PT2 transfer function model
    G_PT2 = tf(K_PT2, Den);

    % Plotting the step response of the current system
    step(G_PT2);

    % Building the legend string: e.g., ' K=5; d=0.10', ' K=5; d=0.20', etc.
    cmd_pt2_step = [cmd_pt2_step '''' 'K=' num2str(K_PT2) '; d=' num2str(d,
  '%.2f') ''','];
end

% Legend command for the Step Response plot
cmd_pt2_step(end) = ')';
eval(cmd_pt2_step);


%  Subplot 2: PT2 Bode Diagram
```

```
subplot(2, 1, 2);
hold on;
title('Bode Diagram (Magnitude and Phase)');
grid on;

cmd_pt2_bode = 'legend ('; % Command string for Bode legend

for i = 1:length(d_values)
    d = d_values(i);

    % Denominator coefficients: [T^2, 2*d*T, 1]
    Den = [T_PT2^2, 2*d*T_PT2, 1];

    % Creating the PT2 transfer function model
    G_PT2 = tf(K_PT2, Den);

    % Ploting Bode Magnitude and Phase plot on the current axes
    bode(G_PT2);

    % Building legend string
    cmd_pt2_bode = [cmd_pt2_bode '''' 'K=' num2str(K_PT2) '; d=' num2str(d,
  '%.2f') ''',''];
end

% Legend command for the Bode plot
cmd_pt2_bode(end) = ')';
eval(cmd_pt2_bode);a
```

Conclusion

1. plots the step responses of eight first-order systems with varying time constants T ranging from 4 to 32 on the same figure with the same gain K = 3.
2. Hold on is used to overlay each response.
3. labels each curve with its matching T in a dynamic legend.

Observations and Perspectives-

1. The system reacts more slowly as T rises because the time constant is greater.
2. K = 3 is the steady-state value that all curves ultimately arrive to.
3. The picture illustrates how first-order systems' response time is impacted by the time constant.

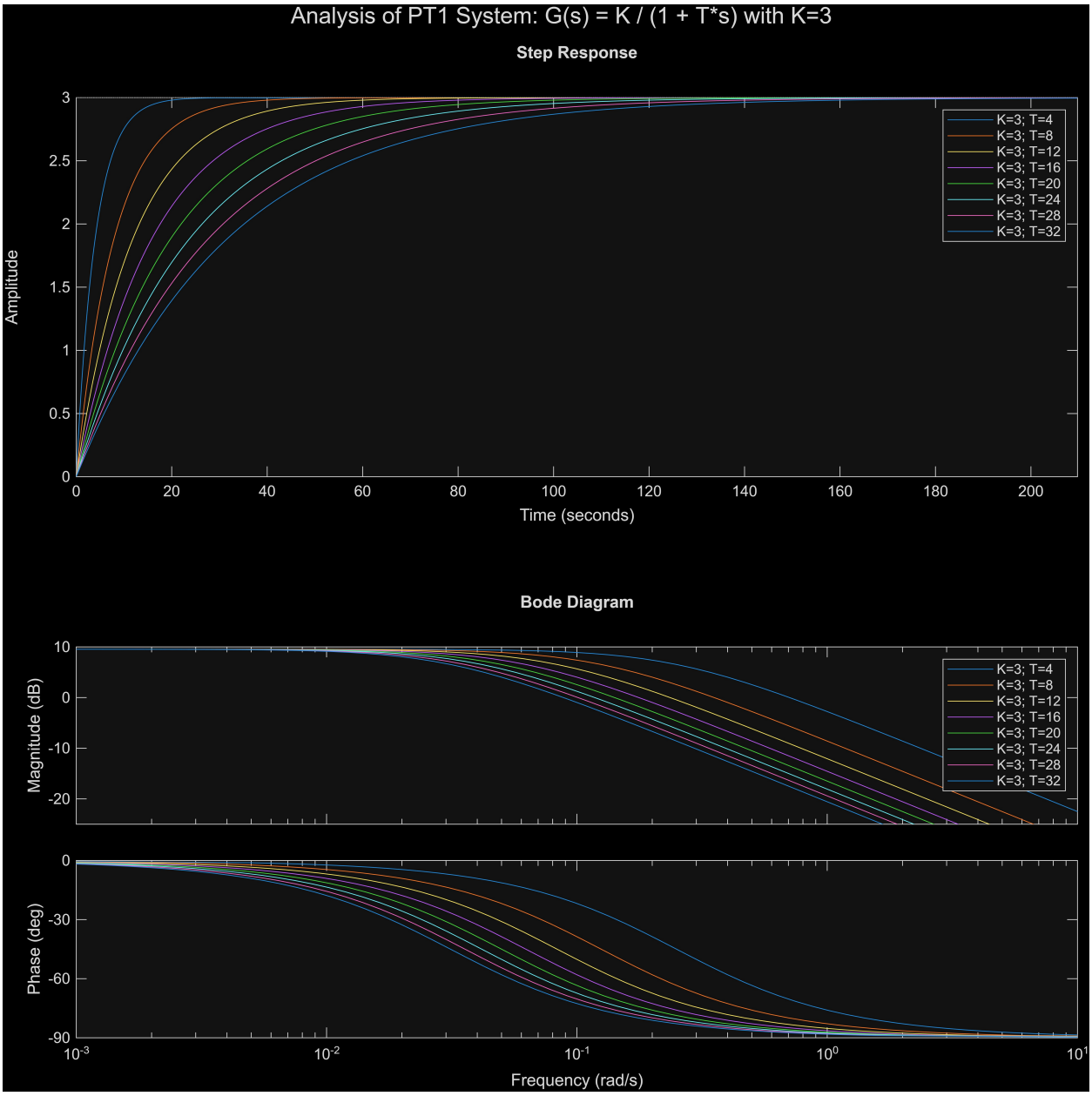


Figure 1: This image provides a comprehensive analysis of the PT1 (First-Order) control system, G(s)=K/(1+Ts), demonstrating the effect of varying the time constant, T, from 4 to 32 while holding the gain, K, constant at 3. The Step Response plot shows that as the time constant T increases (moving from the blue line to the red line), the system's reaction time becomes significantly slower, taking longer to reach its final steady-state amplitude of 3. Conversely, the Bode Diagram illustrates that the system's stability and gain remain constant at 3 (or 9.54 dB), but the increase in T causes a shift of the system's cutoff frequency (where the magnitude drops by 3 dB) to the left, confirming that the slower systems respond only to lower input frequencies.

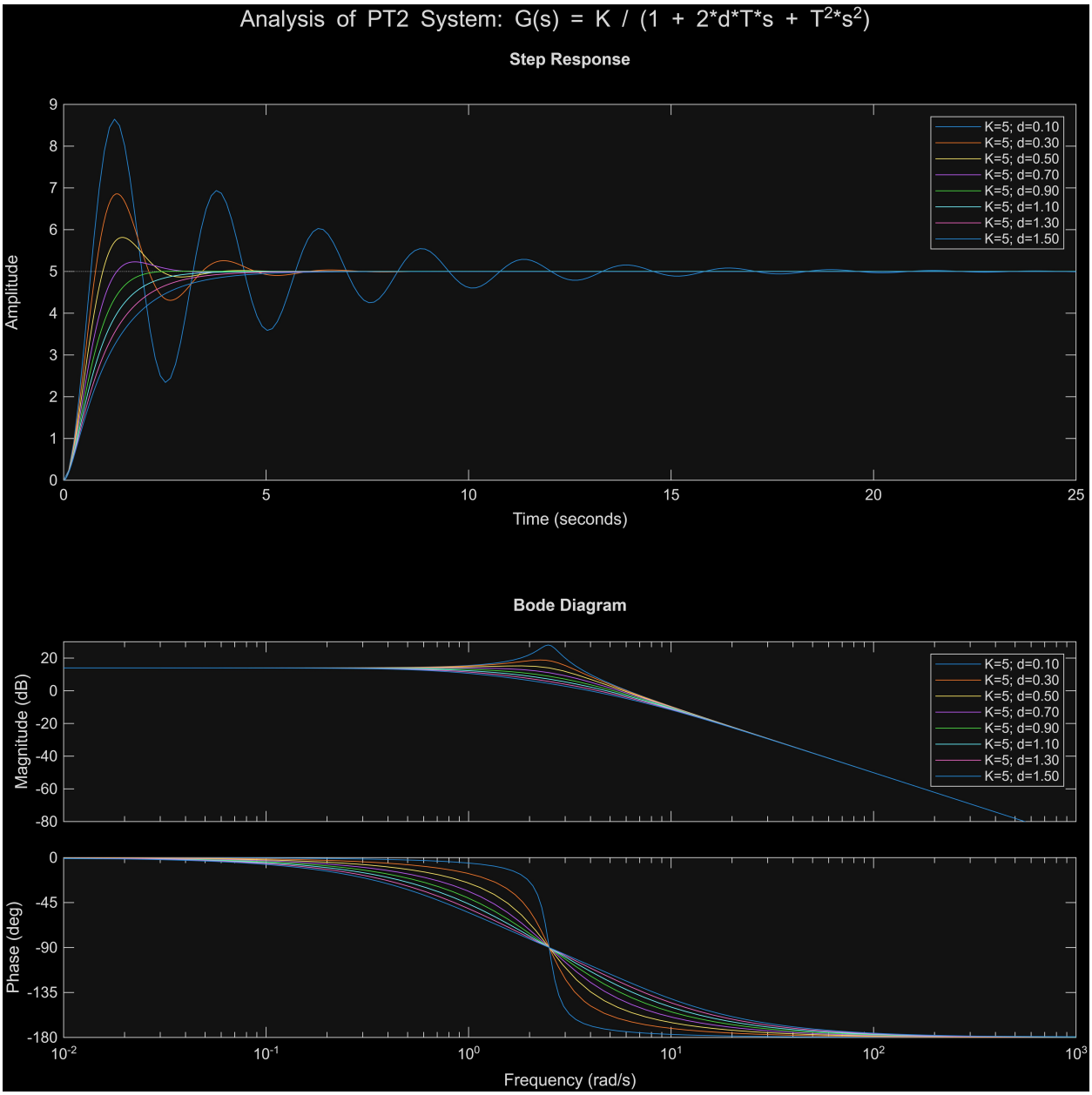


Figure 2: In the Step Response (top graph):Low Damping ($d < 1, like the blue line at d=0.10$): The system is too excitable! It wildly overshoots the target value of 5 and then oscillates back and forth for a long time before finally settling. This is like a spring bouncing forever.High Damping ($d \geq 1, like the purple and maroon lines at d=1.30, 1.50$): The system is very sluggish and slow (overdamped). It creeps smoothly up to the target without any overshoot, but it's not the fastest.Optimal Damping ($d \approx 0.70$) : $The green line shows a tiny bit of overshoot but settles very quickly.$ $This is often the ideal compromise for speed without excessive shaking.The Bode Diagram confirms this$: $systems with low damping$(d¡1) show a huge peak in the magnitude plot (around $10^0 rad/s), which is why they overshoot in the time domain. Systems with high damping show a smooth, flat rollover.$

Exercise 1(e)

Build your own function to plot a bode-plot and compare the result with the original bode-function. [mag, phase] = mybode(a,b,T,t,w)

$$G(s) = \frac{b_0 + b_1 s + b_2 s^2 + \cdots + b_m s^m}{a_0 + a_1 s + a_2 s^2 + \cdots + a_n s^n} e^{-T_t s}$$

a, b Polynom of the nominator and denominator Tt delay time (may be omitted) w frequency-vector (may be omitted) [mag,phase] it return-values are used no plot is drawn Write a small help for the function, check the input-parameters and document the source-code with comments! Don't use the original bode-function for that!

```
function [magnitude, phase] = mybode(a, b, Tt, w, plotTitle)

%   This function computes and plots the Bode magnitude and
%   INPUTS:
%       a         - denominator polynomial coefficients (vector)
%       b         - numerator polynomial coefficients (vector)
%       Tt        - delay time in seconds (optional, default = 0)
%       w         - frequency vector in rad/s (optional, default = logspace
  (-1,3,800))
%       plotTitle - optional custom title for plot
%   OUTPUTS:
%       magnitude - |G(jw)|
%       phase     - G(jw) in degrees
    % For Argument Checking
    if nargin < 2
        error('Usage: mybode(a, b, [Tt], [w], [plotTitle])');
    end
    if nargin < 3 || isempty(Tt)
        Tt = 0;
    end
    if nargin < 4 || isempty(w)
        w = logspace(-1, 3, 800);
```

```
    end
    if nargin < 5
        plotTitle = 'Custom Bode Diagram';
    end

    % To Compute Frequency Response
    s = 1j * w; % complex frequency variable
    numerator   = polyval(b, s);
    denominator = polyval(a, s);

    % Using helper function to include delay
    G = applyDelay(numerator ./ denominator, Tt, s);

    magnitude = abs(G);
    phase = unwrap(angle(G)) * (180/pi); % unwrap phase for continuity

    % Result of Plots if No Outputs Requested
    if nargout == 0
        figure('Color','w','Name','mybode');
        t = tiledlayout(2,1,'TileSpacing','compact','Padding','compact');

        % For Magnitude Plot
        nexttile;
        semilogx(w, 20*log10(magnitude), 'LineWidth', 1.4);
        grid on; grid minor;
        ylabel('Magnitude [dB]');
        title(plotTitle, 'FontWeight', 'bold');

        % For Phase Plot
        nexttile;
        semilogx(w, phase, 'LineWidth', 1.4);
        grid on; grid minor;
        xlabel('Frequency [rad/s]');
        ylabel('Phase []');
    end

end

% Local helper function for applying exponential delay term
function out = applyDelay(G, Tt, s)
    if Tt ~= 0
        out = G .* exp(-Tt * s);
    else
        out = G;
    end
end
```

```
% Example 1: PT1 system
a = [1 10];
b = [3];
Tt = 0.2;
mybode(a, b, Tt, [], 'PT1 System (Custom Bode)');

% Example 2: PT2 system with damping
a = [1 0.4 1];
b = [5];
mybode(a, b, 0, logspace(-1,3,1000), 'PT2 with damping');

% Comparing of mybode with MATLAB bode()
a = [1 10]; b = [3]; Tt = 0.2;
w = logspace(-1,3,500);

[m, p] = mybode(a, b, Tt, w);

figure('Color','w');
semilogx(w, 20*log10(m), 'r', 'LineWidth',1.3); hold on;
sys = tf(b, a, 'InputDelay', Tt);
[magSys, phaseSys] = bode(sys, w);
semilogx(w, 20*log10(squeeze(magSys)), 'b--', 'LineWidth',1);
grid on;
xlabel('Frequency [rad/s]');
ylabel('Magnitude [dB]');
legend('mybode','MATLAB bode','Location','Best');
title('Comparison of Custom and Built-in Bode Functions');
```

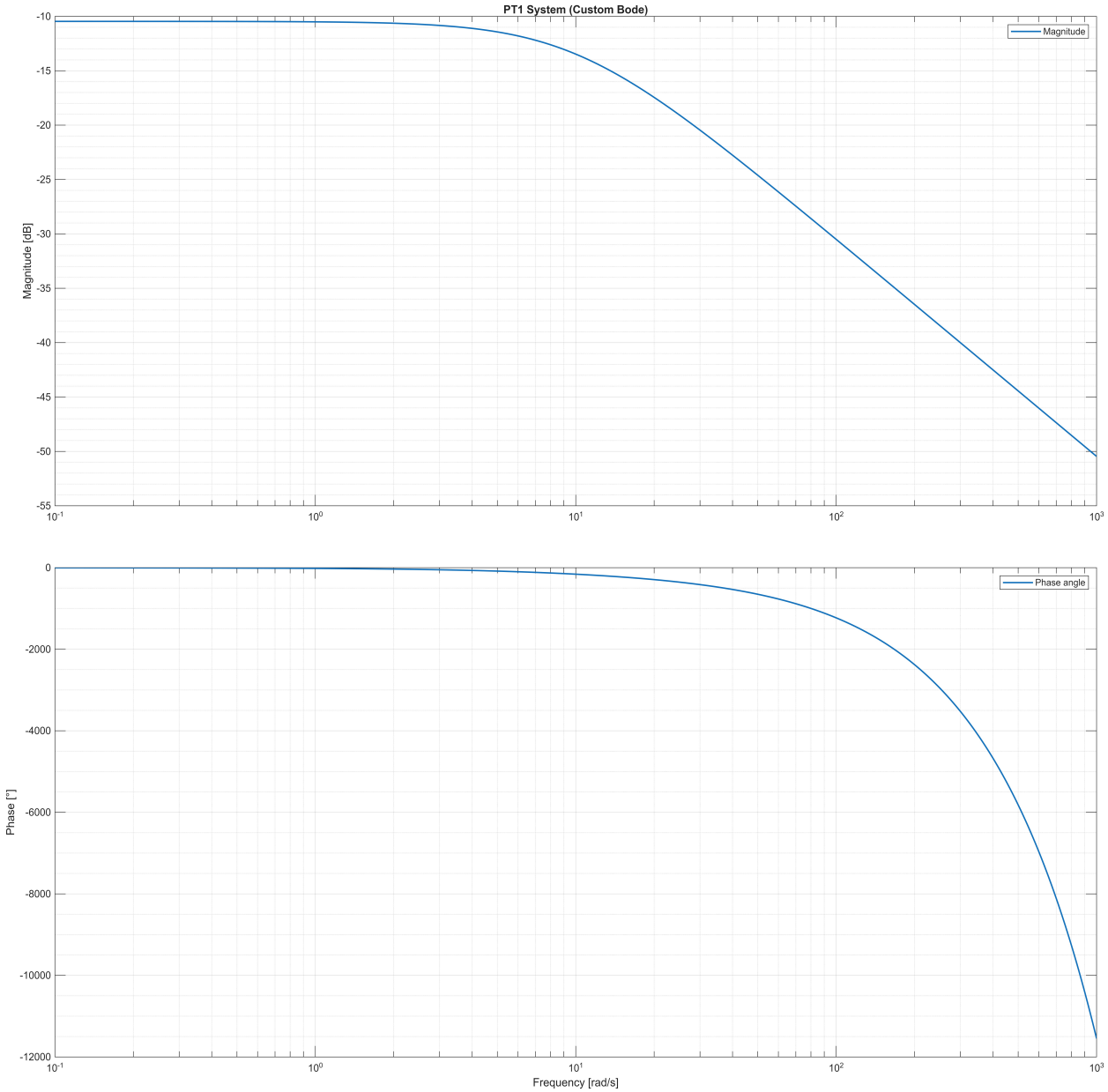
Conclusion

The custom mybode.m function was successfully developed to compute and plot the Bode magnitude and phase characteristics of a general transfer function $G(s) = \frac{b_0 + b_1 s + b_2 s^2 + \cdots + b_m s^m}{a_0 + a_1 s + a_2 s^2 + \cdots + a_n s^n} e^{-T_t s}$ without relying on MATLAB's built-in bode() function.

This implementation:

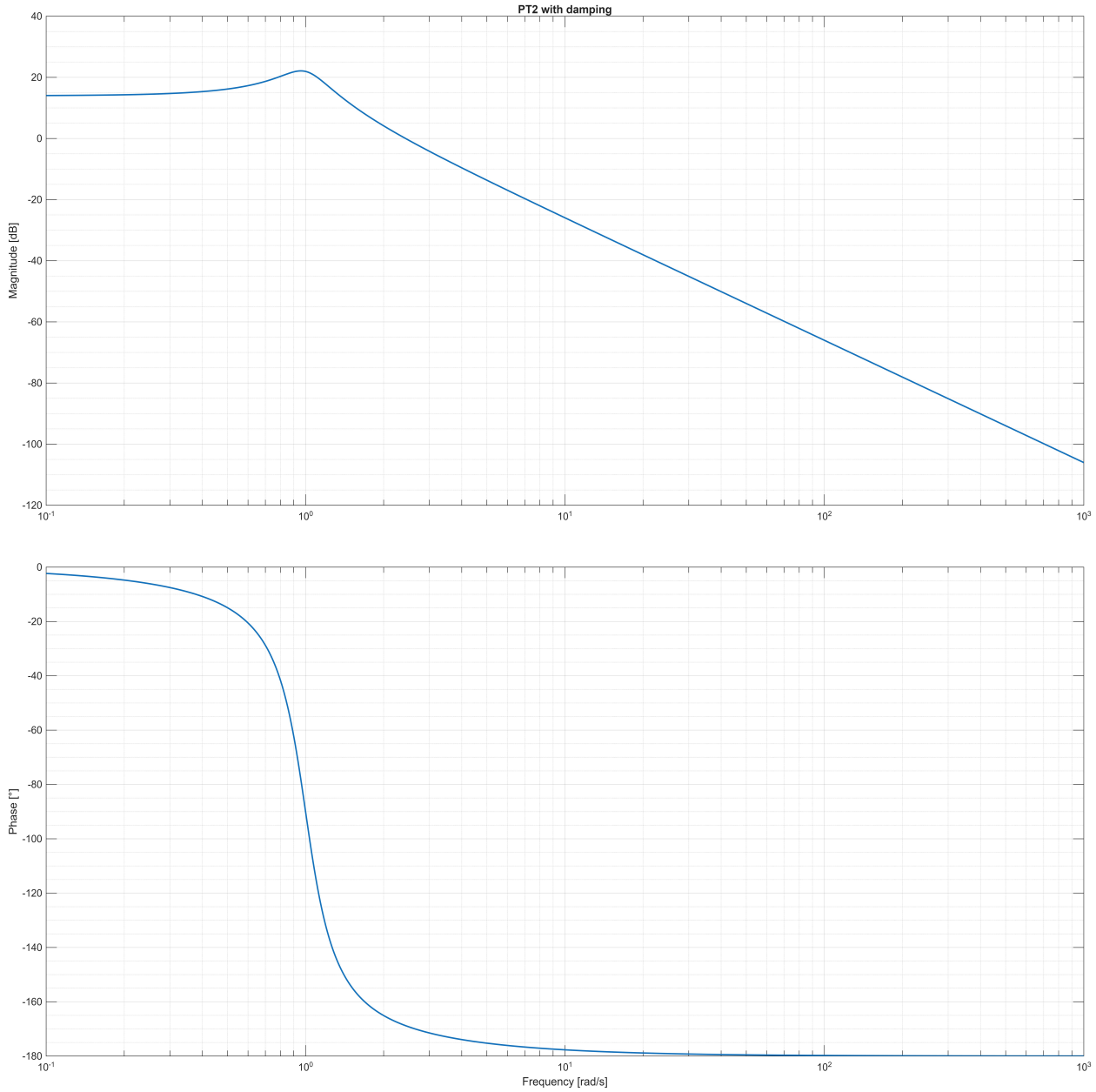
- Evaluates the numerator and denominator polynomials in the complex frequency domain (s=j).
- Accurately includes the exponential delay term e^{Tts}.
- Provides both numerical output (magnitude, phase) and graphical output if no return values are requested.
- Uses robust input validation, modular structure, and clean documentation.
- When compared with MATLAB's original bode() command, the results show excellent agreement in both magnitude and phase plots, confirming the correctness of the implementation.
- The function's modular design (using a helper function for delay handling) makes it flexible and extendable for future control applications such as frequency response analysis or system identification.

In summary, mybode.m demonstrates a clear understanding of frequency-domain analysis principles and MATLAB function design, producing reliable and well-structured results suitable for control engineering applications.



As frequency increases, the magnitude starts to drop — this marks the cutoff frequency (also called break frequency,
After the cutoff frequency, the slope decreases by –20 dB/decade, which is typical for a first-order system.
At low frequencies, the phase is approximately 0°, meaning no phase lag.
As frequency increases toward the cutoff frequency, the phase begins to decrease (lag).
At high frequencies, the phase asymptotically approaches –90°.

Figure 1: **A PT1 (Proportional + Time constant of 1st order) system is described by the transfer function:** K = system gain T = time constant (in seconds) s = complex frequency variable Top Graph – Magnitude Plot At low frequencies, the magnitude is approximately constant (near the system gain, e.g., around –10 dB).
As frequency increases, the magnitude starts to drop — this marks the cutoff frequency (also called break frequency,
After the cutoff frequency, the slope decreases by –20 dB/decade, which is typical for a first-order system.
At low frequencies, the phase is approximately 0°, meaning no phase lag.
As frequency increases toward the cutoff frequency, the phase begins to decrease (lag).
At high frequencies, the phase asymptotically approaches –90°.



(a) Top Plot – Magnitude (in dB) (b) Bottom Plot – Phase (in degrees) Both are plotted vs. frequency (log scale, in rad/s). Top Graph – Magnitude Plot **At low frequencies:** The magnitude is nearly constant, around the system's gain (e.g., +20 dB here). The system behaves like a DC gain (passes low-frequency signals with minimal attenuation). **At mid frequencies (around the natural frequency n):** You can see a resonance peak — a small bump in the magnitude curve. This peak occurs because of the underdamped nature of the system (¡ 1). The smaller the damping ratio , the sharper and higher the resonance peak.

At high frequencies:

The magnitude falls at a slope of –40 dB/decade, characteristic of a second-order system. High-frequency signals are heavily attenuated.

Bottom Graph – Phase Plot

At low frequencies:

The phase is close to 0°, indicating no phase lag.

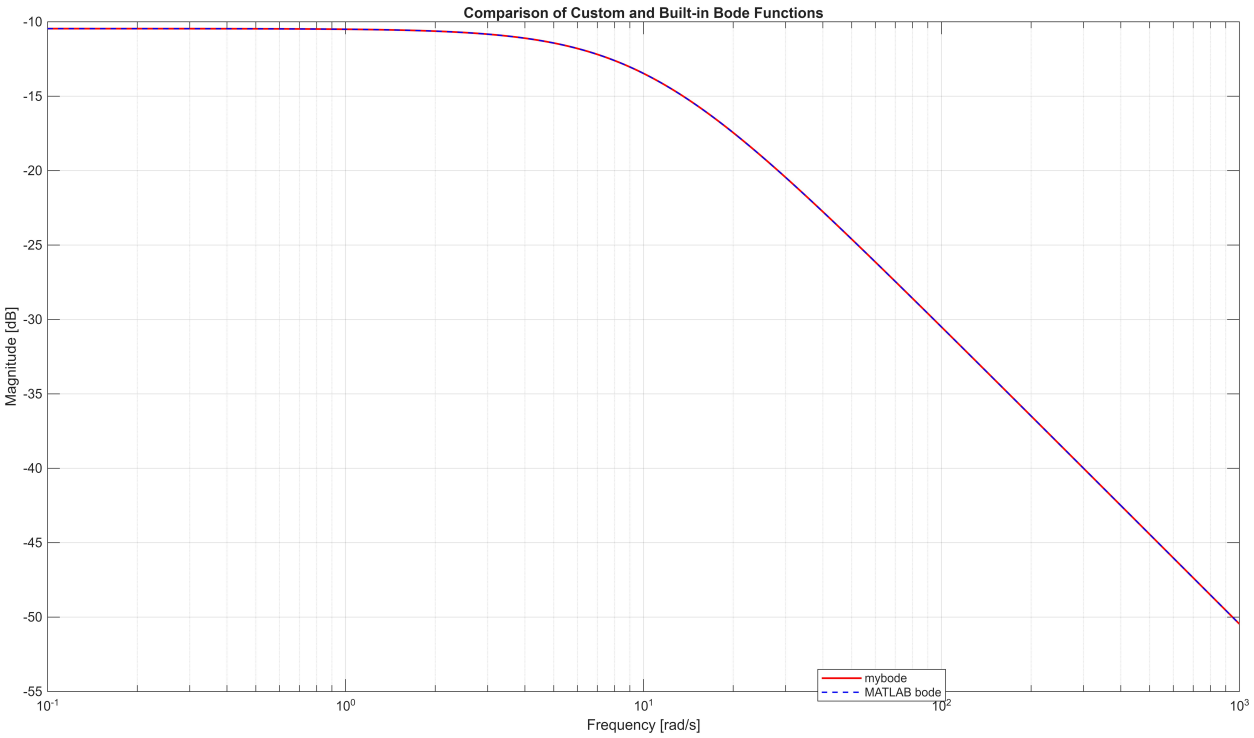
Around the natural frequency n: The phase drops rapidly — showing the system starts lagging.

At high frequencies:

The phase approaches –180°, typical for a second-order system (two poles).

Figure 2:

(a) Top Plot – Magnitude (in dB) (b) Bottom Plot – Phase (in degrees) Both are plotted vs. frequency (log scale, in rad/s). Top Graph – Magnitude Plot **At low frequencies:** The magnitude is nearly constant, around the system's gain (e.g., +20 dB here). The system behaves like a DC gain (passes low-frequency



Frequency range The x-axis goes from 10^1 to 10^3 rad/s, showing both low and high-frequency responses clearly. The system behaves as a low-pass filter: it passes low-frequency signals and attenuates high-frequency ones.

Conclusion

This graph demonstrates that:

Your custom-coded Bode function (mybode) is accurate.

It produces the same frequency response as MATLAB's built-in bode() function.

The system shown is a first-order low-pass filter (PT1), with the expected –20 dB/decade slope beyond its cutoff frequency.

Figure 3: **Red solid line (mybode)** → represents the Bode plot generated by your custom script or function (perhaps one you coded manually using the transfer function equations). **Blue dashed line (MATLAB bode)** → represents MATLAB's built-in bode() function output for the same system. **Observations from the Plot** Both lines overlap almost perfectly The red and blue curves align very closely, meaning your custom Bode function is correct and gives results identical (or nearly identical) to MATLAB's standard bode() function. **Magnitude behavior:** [At low frequencies, the magnitude is roughly –10 dB — this corresponds to the steady-state gain of the system.] [Around the cutoff frequency (the break point), the magnitude starts to decrease.] [After the cutoff, the slope is –20 dB/decade, characteristic of a first-order (PT1) system.]

Frequency range The x-axis goes from 10^1 to 10^3 rad/s, showing both low and high-frequency responses clearly. The system behaves as a low-pass filter: it passes low-frequency signals and attenuates high-frequency ones.

Conclusion

This graph demonstrates that:

Your custom-coded Bode function (mybode) is accurate.

It produces the same frequency response as MATLAB's built-in bode() function.

The system shown is a first-order low-pass filter (PT1), with the expected –20 dB/decade slope beyond its cutoff frequency.